

Day 2 — From Python to Data: NumPy, Pandas & First Look at Data for ML

2 hours | Follows Day 1 (Python basics) | Goal: students can load, inspect, clean, and explore a dataset before we touch any ML algorithm

Lecture Overview

Time	Topic
0:00 - 0:10	Recap Day 1 + Why we need NumPy/Pandas for ML
0:10 - 0:35	NumPy: arrays, why they beat lists, basic operations
0:35 - 1:00	Pandas: Series & DataFrames
1:00 - 1:10	Break
1:10 - 1:35	Pandas: reading CSVs, filtering, cleaning data
1:35 - 1:50	Matplotlib: quick visualization
1:50 - 2:00	Hands-on mini exercise + Wrap-up

1. Recap + Motivation (10 min)

Quick verbal recap — ask, don't tell: - "What's a list? What's a dictionary?" - "How do you write a function?"

Then bridge to today:

"In real ML work, 90% of your time is spent handling data — not building models. Python lists are too slow and clumsy for large datasets, that's why we use **NumPy** (fast math on numbers) and **Pandas** (spreadsheet-like data handling). Every ML pipeline starts with these two."

Say this plainly — it sets up *why* today matters, not just *what* we're learning.

2. NumPy: Arrays (25 min)

Why not just use lists?

Say: "Lists can hold anything, but they're slow for math. NumPy arrays are fixed-type and built for fast, vectorized calculations — this matters a LOT once your datasets have thousands/millions of rows."

Setup (mention to install if needed):

```
pip install numpy
```

Creating arrays

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)
```

```
print(type(a))

# 2D array (like a small table)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix)
print(matrix.shape)  # (2, 3) -> 2 rows, 3 columns
```

Why arrays are better — vectorized math (this is the “wow” moment)

```
nums = [1, 2, 3, 4, 5]
# In plain Python you'd need a loop to multiply every element:
doubled = [n * 2 for n in nums]

# With NumPy, no loop needed:
arr = np.array([1, 2, 3, 4, 5])
doubled = arr * 2
print(doubled)  # [2 4 6 8 10]
```

Live-code this side by side — it’s the clearest way to show *why* NumPy exists.

Useful array operations

```
arr = np.array([10, 20, 30, 40, 50])

print(arr.mean())  # average
print(arr.sum())   # total
print(arr.max())   # largest
print(arr.min())   # smallest
print(arr[0])      # indexing – same as lists
print(arr[1:3])    # slicing – same as lists
```

Teaching tip: Point out `mean()`, `sum()`, `max()` — students will use these constantly once we get to ML (e.g. averaging errors, normalizing data).

Quick check-in question for students

“If I have exam scores [55, 70, 90, 40, 85], how would you find the average using NumPy?” (Let them try:
`np.array([55,70,90,40,85]).mean()`)

3. Pandas: Series & DataFrames (25 min)

Say: “NumPy is great for numbers. But real data has labels — names, dates, categories. That’s what Pandas is for. Think of it as Excel inside Python.”

```
pip install pandas
```

Series (a single labeled column)

```
import pandas as pd

ages = pd.Series([21, 25, 19, 30], name="age")
print(ages)
```

DataFrame (a full table)

```
data = {
    "name": ["Alice", "Bob", "Charlie", "Diana"],
    "age": [21, 25, 19, 30],
    "score": [85, 72, 90, 68]
```

```

}

df = pd.DataFrame(data)
print(df)

```

Show the output — students should visually connect this to a spreadsheet.

Basic exploration (the commands they'll use every single time)

```

print(df.head())           # first 5 rows
print(df.shape)            # (rows, columns)
print(df.columns)          # column names
print(df.describe())       # quick stats: mean, min, max, etc.
print(df["age"])            # select one column
print(df[["name", "age"]]) # select multiple columns

```

Teaching tip: `df.describe()` is a great “wow” moment — one line gives mean/std/min/max for all numeric columns. Emphasize this will be their go-to first command on any new dataset.

Break (10 min)

Let them stretch. Good moment for 1:1 questions or letting them try `df.describe()` on their own made-up data.

4. Pandas: Reading CSVs, Filtering & Cleaning (25 min)

Reading real data

```

df = pd.read_csv("students.csv")
print(df.head())

```

(If you don't have a sample CSV, create one live using the DataFrame from before and save it: `df.to_csv("students.csv", index=False)` — then re-read it. Great way to show the full loop.)

Filtering rows (very important for ML — students WILL use this constantly)

```

# Students who scored above 80
high_scorers = df[df["score"] > 80]
print(high_scorers)

# Combine conditions
result = df[(df["score"] > 70) & (df["age"] < 25)]
print(result)

```

Handling missing data (real-world data is messy — always)

```

# Suppose some scores are missing (NaN)
df_missing = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "score": [85, None, 90]
})

print(df_missing.isnull())           # shows True where data is missing
print(df_missing.dropna())           # removes rows with missing data

```

```
print(df_missing.fillna(0)) # fills missing values with 0
```

Say clearly: “Every ML model breaks if you feed it missing values. Cleaning data like this is not optional — it’s step one, always.”

Adding a new column (common ML prep step)

```
df["pass"] = df["score"] > 60 # True/False column
print(df)
```

5. Matplotlib: Quick Visualization (15 min)

Say: “Before building any ML model, you should *look* at your data. A quick chart reveals patterns numbers hide.”

```
pip install matplotlib

import matplotlib.pyplot as plt

df = pd.DataFrame({
    "name": ["Alice", "Bob", "Charlie", "Diana"],
    "score": [85, 72, 90, 68]
})

plt.bar(df["name"], df["score"])
plt.xlabel("Student")
plt.ylabel("Score")
plt.title("Student Scores")
plt.show()
```

Also show a simple histogram (useful for distributions — a core ML concept):

```
scores = [55, 70, 90, 40, 85, 60, 75, 95, 50, 88]
plt.hist(scores, bins=5)
plt.title("Distribution of Scores")
plt.show()
```

Teaching tip: Don’t go deep into Matplotlib syntax today — the goal is just “I can look at my data visually.” We’ll return to this when we cover model evaluation.

6. Hands-on Practice (10-15 min)

Give this task, let them build it live, walk around and help:

“Create a DataFrame with 5 students: name, age, and 2 test scores. Then: 1. Add a column average = mean of the two scores 2. Filter and print only students with average above 70 3. Plot a bar chart of names vs. average score”

Sample solution (keep hidden until after they attempt it):

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "name": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "age": [21, 22, 20, 23, 21],
    "test1": [80, 65, 90, 55, 75],
    "test2": [85, 70, 95, 60, 80]
}
df = pd.DataFrame(data)
```

```
df["average"] = (df["test1"] + df["test2"]) / 2

top_students = df[df["average"] > 70]
print(top_students)

plt.bar(df["name"], df["average"])
plt.title("Average Scores")
plt.show()
```

7. Wrap-Up & Q&A (10 min)

Recap out loud: - **NumPy** → fast math on arrays, no loops needed -
Pandas → tables (DataFrames), filtering, cleaning missing data -
Matplotlib → quick visual look at data before modeling

Say: “Every single ML project — no exceptions — starts with these steps: load data, clean it, look at it. Tomorrow/next class we start using this data to actually build a model.”

Homework: Find (or make up) a small CSV with at least 3 columns (e.g. product name, price, rating). Load it with Pandas, print `.describe()`, filter for one condition, and make one chart.

Tips for Teaching This Session

- Live-code everything — don’t paste finished code, type it out so they see the process (and typos/errors, which is realistic).
- Constantly connect back to ML: “we’ll use this exact function when we build our first model” — keeps motivation high even though this feels like “just data handling.”
- `df.describe()` and filtering with `df[condition]` are the two things to over-emphasize — they’ll use these in almost every future class.
- If short on time, cut the histogram example first, then the missing-data section — filtering and `describe()` are the non-negotiables.
- Keep a sample CSV ready beforehand in case `read_csv` demo needs a real file instead of one built on the fly.